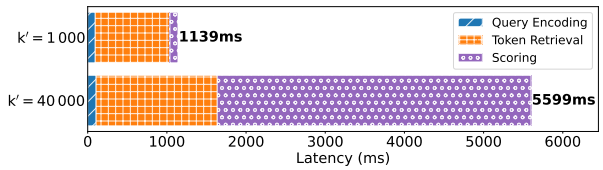
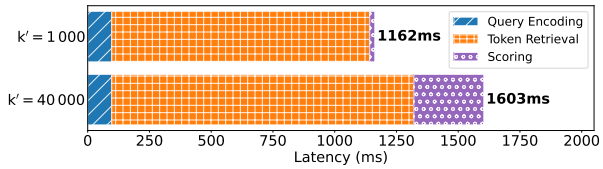


(a) Latency breakdown of the reference implementation.



(b) Latency breakdown of the optimized implementation.

Figure 3: Breakdown of XTR_{base}/ScaNN on LoTTE Pooled.

As seen in Figure 3a, the scoring stage constitutes a significant bottleneck in the end-to-end latency of the XTR framework, particularly when dealing with large values of k'. We argue that this performance bottleneck is largely attributed to an *unoptimized* implementation in the released code, which relies on native Python data structures and manual iteration, introducing substantial overhead, especially for large numbers of token embeddings. We refactor this implementation to leverage optimized data structures and vectorized operations. This helps us uncover hidden performance inefficiencies and establish a baseline for further optimization. Our improved XTR implementation is publicly available in the corresponding GitHub repository[3].

We present the evaluation of our optimized implementation in Figure 3b. Notably, the optimized implementation's end-to-end latency is significantly lower than that of the reference implementation ranging from an end-to-end 3.5x speedup on LoTTE pooled to a 6.3x speedup on LoTTE Lifestyle for $k = 1\,000$. This latency reduction is owed in large parts to a more efficient *scoring* implementation – a 14x speedup on LoTTE Pooled. In particular, this reveals token retrieval as the fundamental bottleneck of the XTR framework. Although the optimized scoring stage accounts for a small fraction of the overall end-to-end latency, it is still slow in absolute terms, ranging from 33ms to 281ms for $k = 1\,000$ on BEIR NFCorpus and LoTTE Pooled, respectively.

4 WARP

WARP optimizes retrieval for the refined late interaction architecture introduced in XTR. Seeking to find the best of both the XTR and PLAID worlds, **WARP introduces the novel $\text{WARP}_{\text{SELECT}}$ algorithm for candidate generation, which effectively avoids gathering token-level representations, and proposes an optimized two-stage reduction for faster scoring via a dedicated C++ kernel combined with implicit decompression.** WARP also uses specialized inference runtimes for faster query encoding.

[3]https://github.com/jlscheerer/xtr-eval

As in XTR, queries and documents are encoded *independently* into embeddings at the token-level using a fine-tuned T5 transformer [16]. To scale to large datasets, document representations are computed in advance and constitute WARP's index. The similarity between a query q and document d is modeled using XTR's adaptation of ColBERT's summation of MaxSim operations[4]:

$$S_{d,q} = \sum_{i=1}^{n} \max_{1 \le j \le m} [\hat{\mathbf{A}}_{i,j} q_i^T d_j + (1 - \hat{\mathbf{A}}_{i,j}) m_i] \tag{1}$$

where q and d are the matrix representations of the query and passage embeddings respectively, m_i denotes the missing similarity estimate for q_i, and $\hat{\mathbf{A}}$ describes XTR's alignment matrix. In particular, $\hat{\mathbf{A}}_{i,j} = \mathbb{1}_{[j \in \text{top-}k'_j(\mathbf{d}_{i,j'})]}$ captures whether a document token embedding of a candidate passage was retrieved for a specific query token q_i as part of the token retrieval stage. We refer to [8, 11] for an intuition behind this choice of scoring function.

4.1 Index Construction

Akin to ColBERTv2 [19], WARP's compression strategy involves applying k-means clustering to the produced document token embeddings. As in ColBERTv2, we find that using a sample of all passages proportional to the square root of the collection size to generate this clustering performs well in practice. After having clustered the sample of passages, all token-level vectors are encoded and stored as quantized residual vectors to their nearest cluster centroid. Each dimension of the quantized residual vector is a b-bit encoding of the delta between the centroid and the original uncompressed vector. In particular, these deltas are stored as a sequence of $128 \cdot \frac{b}{8}$ 8-bit values, wherein 128 represents the transformer's token embedding dimension. Typically, we set $b = 4$, i.e., compress each dimension of the residual into a single nibble, for an $8x$ compression[5]. In this case, each compressed 8-bit value stores 2 indices in the range $[0, 2^b)$. Instead of quantizing the residuals uniformly, WARP uses quantiles derived from the empirical distribution to determine bucket boundaries and the corresponding representative values. This process allows WARP to allocate more quantization levels to densely populated regions of the data distribution, thereby minimizing the overall quantization error for residual compression.

4.2 Retrieval

Extending PLAID, the retrieval process in the WARP engine is divided into four distinct steps: query encoding, candidate generation, decompression, and scoring. Figure 4 illustrates the retrieval process in WARP. The process starts by encoding the query text into q, a (query_maxlen, 128)-dimensional tensor, using the underlying Transformer model.[6] Next, the most similar n_{probe} centroids are identified for each of the query_maxlen query token embeddings.

Subsequently, WARP identifies all document token embeddings belonging to the clusters of the selected centroids and computes their *individual* relevance score. Computing this score involves

[4]Note that we omit the normalization via $\frac{1}{Z}$, as we are only interested in the relative ranking between documents and the normalization constant is identical for any retrieved document.

[5]As compared to an uncompressed 32-bit floating point number per dimension.

[6]We set query_maxlen = 32 in accordance with the XTR paper.

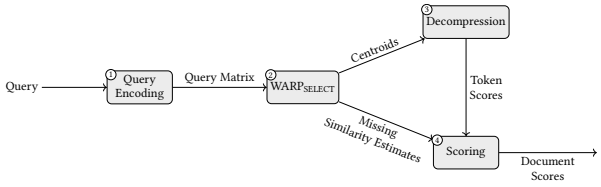


Figure 4: WARP Retrieval consisting of query encoding, WARP$_{\text{SELECT}}$, decompression, and scoring. Notably, centroid selection is combined with the computation of missing similarity estimates in WARP$_{\text{SELECT}}$.

decompressing residuals of the identified document token embeddings and calculating their cosine similarity with the relevant query token embedding. Finally, WARP implicitly constructs an $n_{\text{candidates}} \times \text{query\_maxlen}$ score matrix S[7], where each entry S_{d_i,q_j} contains the maximum retrieved score for the i-th candidate passage d_i and the j-th query token embedding q_j:

$$\max_{1 \le j \le m} \hat{\mathbf{A}}_{i,j} q_i^T d_j$$

Matrix entries not populated during token retrieval, i.e., $\hat{\mathbf{A}}_{i,j} = 0$, are imputed with a missing similarity estimate, as postulated in the XTR framework. To compute the relevance score of a document d_i, the cumulative score over all query tokens is computed: $\sum_j S_{d_i,q_j}$. To produce the ordered set of passages, the set of scores is sorted and the top k highest scoring passages are returned.

4.3 WARP$_{\text{SELECT}}$

In contrast to ColBERT, which populates the entire score matrix for the items retrieved, XTR only populates the score matrix with scores computed as part of the token retrieval stage. To account for the contribution of any missing tokens, XTR relies on *missing similarity imputation*, in which they set any missing similarity of the query token for a specific document as the lowest score obtained as part of the token retrieval stage. The authors argue that this approach is justified as it constitutes a *natural* upper bound for the true relevance score. In the case of WARP, this bound is no longer guaranteed to hold.[8]

Instead, WARP defines a novel strategy for missing similarity imputation based on cumulative cluster sizes, WARP$_{\text{SELECT}}$. Given the query embedding matrix q and the list of centroids C (Section 4.1), WARP computes the token-level query-centroid relevance scores. As both the query embedding vectors and the set of centroids are normalized, the cosine similarity scores $S_{c,q}$ can be computed efficiently as a matrix product:

$$S_{c,q} = C \cdot q^T$$

Once these relevance scores have been computed, WARP identifies the n_{probe} centroids with the largest similarity scores for decompression, as part of candidate generation.

Using these query–centroid similarity scores, WARP$_{\text{SELECT}}$ folds the estimation of missing similarity scores into candidate generation. Specifically, for each query token q_i, it sets m_i from Equation (1) as the first element in the sorted list of centroid scores for which the cumulative cluster size exceeds a threshold t'. This method is particularly attractive as all the centroid scores have already been computed and sufficiently sorted as part of candidate generation, so the cost of computing missing similarity imputation with this method is negligible.

We find that t' is easy to configure (see Section 4.6) without compromising retrieval quality or efficiency. This represents a significant improvement over the missing similarity estimate used in the XTR reference implementation, where the estimate is *inherently* tied to the number of retrieved tokens. Intuitively, increasing k' in XTR may only help refine the missing similarity estimate, but not necessarily increase the density of the score matrix.[9]

4.4 Decompression

The input for the decompression phase is the set of n_{probe} centroid indices for each of the `query_maxlen` query tokens. Its goal is to calculate relevance scores between each query token and the embeddings within the identified clusters. For a query token q_i, let $c_{i,j}$, where $j \in [n_{\text{probe}}]$, be the set of centroid indices identified during candidate generation. Let $r_{i,j,k}$ be the set of residuals associated with cluster $c_{i,j}$. The decompression step computes:

$$s_{i,j,k} = \text{decompress}(C[c_{i,j}], r_{i,j,k}) \times q_i^T \;\; \forall \; i, j, k \tag{2}$$

The decompress function converts residuals from their compact representation into uncompressed vectors. Each residual $r_{i,j,k}$ is composed of 128 indices, each b bits wide. These indices reference values in the bucket weights vector $\omega \in \mathbb{R}^{2^b}$ and are used to offset the centroid $C[c_{i,j}]$. The decompress function is defined as:

$$\text{decompress}(C[c_{i,j}], r_{i,j,k}) = C[c_{i,j}] + \sum_{d=1}^{128} \boldsymbol{e}_d \cdot \omega[(r_{i,j,k})_d] \tag{3}$$

Here, $\boldsymbol{e}_d$ is the unit vector for dimension d, and $\omega[(r_{i,j,k})_d]$ is the weight value at index $(r_{i,j,k})_d$ for dimension d. In other words, the indices are used to look up specific entries in ω for each dimension independently, adjusting the centroid accordingly.

WARP avoids *explicitly* decompressing residuals, as PLAID does, by leveraging the observation that the scoring function decomposes between centroids and residuals. As a result, WARP reuses the query-centroid relevance scores $S_{c,q}$, computed as part of candidate generation. That is, observe that:

$$\begin{aligned} s_{i,j,k} &= \text{decompress}(C[c_{i,j}], r_{i,j,k}) \times q_i^T \\ &= (C[c_{i,j}] \times q_i^T) + (\sum_{d=1}^{128} \omega[(r_{i,j,k})_d] q_{i,d}) \end{aligned} \tag{4}$$

To accelerate decompression, WARP computes $v = \hat{q} \times \hat{\omega}$, wherein $\hat{q} \in \mathbb{R}^{\text{query\_maxlen} \times 128 \times 1}$ represents the query matrix that has been *unsqueezed* along the last dimension, and $\hat{\omega} \in \mathbb{R}^{1 \times 2^b}$ denotes the vector of bucket weights that has been *unsqueezed* along the first dimension.

[7] Note that our optimized implementation does not physically construct this matrix.
[7] For XTR baselines, 'candidate generation' refers to the token retrieval stage.
[8] Even in XTR's case, the bound is only approximate, as ScaNN does not guarantee exact nearest neighbors.
[9] This is because tokens retrieved with a larger k' are often from new documents and, therefore, do not refine the scores of already retrieved ones.

With these definitions, WARP can decompress and score candidate tokens via:

$$\begin{aligned} s_{i,j,k} &= S_{c_j,q_i} + \sum_{d=1}^{128} (\omega \cdot q_{i,d})[(r_{i,j,k})_d] \\ &= S_{c_j,q_i} + \sum_{d=1}^{128} v_{i,d}[(r_{i,j,k})_d] \end{aligned} \tag{5}$$

Note that candidate scoring can now be implemented as a simple *selective sum*. As the bucket weights are shared among centroids and the query-centroid relevance scores have already been computed during candidate generation, WARP can decompress and score arbitrarily many clusters using $O(1)$ multiplications. **This refined scoring function is far more efficient than the one outlined in PLAID,[10] as it never computes the decompressed embeddings explicitly and instead directly emits the resulting candidate scores.** We provide an efficient implementation of the selective sum of Equation (4) and realize unpacking of the residual representation using low-complexity bitwise operations as part of WARP's C++ kernel for decompression.

4.5 Scoring

At the end of the decompression phase, we have `query_maxlen` $\times$ n_{probe} strides of decompressed candidate document *token-level scores* and their corresponding document identifiers. Scoring combines these scores with the missing similarity estimates, computed during candidate generation, to produce *document-level scores*. This process corresponds to constructing the score matrix and taking the row-wise sum.

Explicitly constructing the score matrix, as in the reference XTR implementation, introduces a significant bottleneck, particularly for large values of n_{probe}. To address this, WARP efficiently aggregates token-level scores using a two-stage reduction process:

- **Token-level reduction** For each query token, reduce the corresponding set of n_{probe} strides using the `max` operator. This step *implicitly* fills the score matrix with the maximum per-token score for each document. As a single cluster can contain multiple document token embeddings originating from the same document, WARP performs *inner-cluster* `max`-reduction directly during the decompression phase.
- **Document-level reduction** Reduce the resulting strides into document-level scores using a `sum` aggregation. It is essential to handle missing values properly at this stage – any missing per-token score must be replaced by the corresponding missing similarity estimate, ensuring compliance with the XTR scoring function described in Equation (1). This reduction step corresponds to the row-wise summation of the score matrix.

After performing both reduction phases, the final stride contains the document-level scores and the corresponding identifiers for all candidate documents. To retrieve the result set, we perform heap select to obtain the top-k documents, similar to its use in the candidate generation phase.

[10] PLAID cannot adopt this approach directly, as it *normalizes* the vectors after decompression. Empirically, we find that this normalization step has little effect on the final embeddings as the residuals are already normalized prior to quantization.

Formally, we consider a stride S to be a list of key-value pairs:

$$S = \{(k_i, v_i)\};\ \mathrm{K}(S) = \{k_i \mid (k_i, v_i) \in S\};\ \mathrm{V}(S) = \{v_i \mid (k_i, v_i) \in S\}$$

Thus, strides implicitly define a partial function $f_S : K \rightharpoonup V(S)$:

$$f_S(k) = \begin{cases} v_i & \text{if } \exists v_i.\ (k, v_i) \in S \\ \bot & \text{otherwise} \end{cases}$$

We define a reduction as a combination of two strides S_1 and S_2 using a binary function r into a single stride by applying r to values of matching keys:

$$\text{reduce}(r, S_1, S_2) = \{(k, r(f_{S_1}(k), f_{S_2}(k))) \mid k \in K(S_1) \cup K(S_2)\}$$

With these definitions, token-level reduction can be written as:

$$r_{\text{tok}}(v_1, v_2) = \begin{cases} \max(v_1, v_2) & \text{if } v_1 \neq \bot \wedge v_2 \neq \bot \\ v_1 & \text{if } v_1 \neq \bot \wedge v_2 = \bot \\ v_2 & \text{otherwise} \end{cases} \tag{6}$$

Defining document-level reduction is slightly more complex as it involves incorporating the corresponding missing similarity estimates m. After token-level reduction each of the `query_maxlen` strides $S_1, \ldots, S_{\texttt{query\_maxlen}}$ *covers* scores for a single query token q_i. We set $S_{i,i} = S_i$ and define:

$$S_{i,j} = \text{reduce}(r_{\text{doc},(i,k,j)}, S_{i,k}, S_{k+1,j}) \tag{7}$$

for any choice of $i \leq k < j$, wherein $r_{\text{doc},(i,k,j)}$ merges two successive, non-overlapping strides $S_{i,k}$ and $S_{k+1,j}$. The resulting stride, $S_{i,j}$, now covers scores for query tokens $q_i, \ldots, q_j$. Defining $r_{\text{doc},(i,k,j)}$ is relatively straightforward:

$$r_{\text{doc},(i,k,j)}(v_1, v_2) = \begin{cases} v_1 + v_2 & \text{if } v_1 \neq \bot \wedge v_2 \neq \bot \\ v_1 + (\sum_{t=k+1}^{j} m_t) & \text{if } v_1 \neq \bot \wedge v_2 = \bot \\ (\sum_{t=i}^{k} m_t) + v_2 & \text{otherwise} \end{cases} \tag{8}$$

It is easy to verify that $S_{i,j}$ is well-defined, i.e., independent of the choice of k. The result of document-level reduction is $S_{1,\texttt{query\_maxlen}}$ and can be obtained by recursively applying Equation (7) to strides of increasing size.

WARP's two-stage reduction process, along with the final sorting step, is illustrated in Figure 5. In the token-level reduction stage, strides are merged by selecting the maximum value for matching keys. In the document-level reduction stage, values for matching keys are summed, with missing values being substituted by the corresponding missing similarity estimates.

In our implementation, we conceptually construct a binary tree of the required merges and alternate between using two scratch buffers to avoid additional memory allocations. We realize Equation (8) using a prefix sum, which involves precalculating running totals to eliminate the need to compute sums explicitly later on.

4.6 Hyperparameters

In this section, we analyze the effects of WARP's three primary hyperparameters, namely:

- n_{probe} – the #clusters to decompress per query token
- t' – the threshold on the cluster size used for $\text{WARP}_{\text{SELECT}}$
- b – the number of bits per dimension of a residual vector

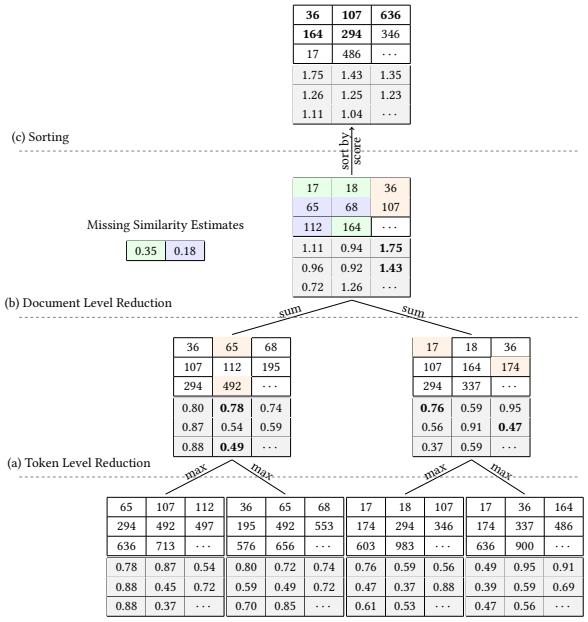


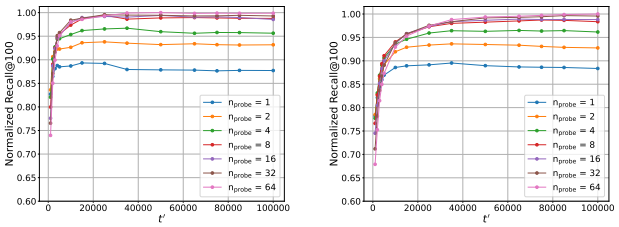
Figure 5: WARP's scoring phase: (a) In token-level reduction, strides are max-reduced. (b) In document-level reduction, values are summed, accounting for missing similarity estimates. (c) Scores are sorted, yielding the top-k results.

To study the effects of n_{probe} and t', we analyze the normalized Recall@100[11] as a function of t' for $n_{\text{probe}} \in \{1, 2, 4, 8, 16, 32, 64\}$ across four development datasets of increasing size: BEIR NFCorpus, BEIR Quora, LoTTE Lifestyle, and LoTTE Pooled. For further details on the datasets, please refer to Table 1. Figure 6 visualizes the results of our analysis. We observe a consistent pattern across all evaluated datasets, namely substantial improvements as n_{probe} increases from 1 to 16 (i.e., 1, 2, 4, 8, 16), followed by only marginal gains in Recall@100 beyond that. A notable exception is BEIR NFCorpus, where we still observe significant improvement when increasing from $n_{\text{probe}} = 16$ to $n_{\text{probe}} = 32$. We hypothesize that this is due to the small number of embeddings per cluster in NFCorpus, limiting the number of scores available for aggregation. Consequently, we conclude that setting $n_{\text{probe}} = 32$ strikes a good balance between end-to-end latency and retrieval quality.

In general, we find that WARP is highly robust to variations in t'. However, smaller datasets, such as NFCorpus, appear to benefit from a smaller t', while larger datasets perform better with a larger t'. Empirically, we find that setting t' proportional to the square root of the dataset size consistently yields strong results across all datasets. Moreover, increasing t' beyond a certain point no longer improves recall, leading us to bound t' by a maximum value, t'_{max}.

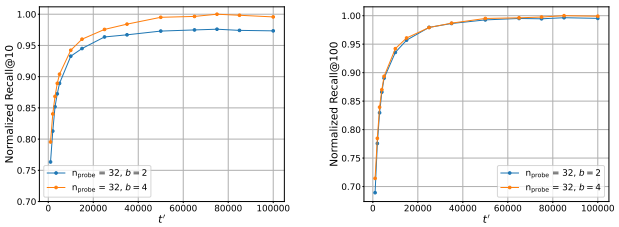
Next, we aim to quantify the effect of b on the retrieval quality of WARP, as shown in Figure 7. To do this, we compute the nRecall@k for $n_{\text{probe}} = 32$ and $k \in \{10, 100\}$ using two datasets: LoTTE Science and LoTTE Pooled.

[11]The normalized Recall@k is calculated by dividing Recall@k by the dataset's maximum, effectively scaling values between 0 and 1 to ensure comparability across datasets.



(a) LoTTE Science (Dev Set) **(b) LoTTE Pooled (Dev Set)**

Figure 6: nRecall@100 as a function of t' and n_{probe}



(a) LoTTE Pooled (Dev Set), nRecall@10 **(b) LoTTE Pooled (Dev Set), nRecall@100**

Figure 7: nRecall@k as a function of t' and b

Dataset		Dev		Test	
		#Queries	#Passages	#Queries	#Passages
BeIR [20]	NFCORPUS	324	3.6K	323	3.6K
	SciFact	–	–	300	5.2K
	SCIDOCS	–	–	1,000	25.7K
	Quora	5,000	522.9K	10,000	522.9K
	FiQA-2018	500	57.6K	648	57.6K
	Touché-2020	–	–	49	382.5K
LoTTE [19]	Lifestyle	417	268.9K	661	119.5K
	Recreation	563	263.0K	924	167.0K
	Writing	497	277.1K	1,071	200.0K
	Technology	916	1.3M	596	638.5K
	Science	538	343.6K	617	1.7M
	Pooled	2,931	2.4M	3,869	2.8M

Table 1: Datasets used for evaluating XTR_{base}/WARP performance. The evaluation includes 6 datasets from BEIR [20] and 6 from LoTTE [19].

Our results show a significant improvement in retrieval performance when increasing b from 2 to 4, particularly for smaller values of k. For larger values of k, the difference in performance diminishes, particularly for the LoTTE Pooled dataset.

5 Evaluation

We now evaluate WARP on six datasets from BEIR [20] and six datasets from LoTTE [19] listed in Table 1. We use servers with 28 `Intel Xeon Gold 6132 @ 2.6 GHz` CPU cores[12] and 500 GB

[12]Each core has 2 threads for a total of 56 threads.